

# Projeto Final 02: Evolução Arquitetural com Design Patterns

Aluno: Jefferson Barzan Alexandrino  
11/06/2024

## 1. Problemas Identificados na Arquitetura Inicial

A primeira versão do sistema foi desenvolvida como um MVP funcional, priorizando rapidez de implementação. Entretanto, com o crescimento da aplicação, alguns problemas arquitetônicos tornaram-se evidentes.

### Centralização Excessiva de Responsabilidades

O arquivo `server.js` era responsável simultaneamente por:

- Inicialização da aplicação;
- Configuração do banco de dados;
- Criação das tabelas;
- Definição das rotas HTTP;
- Execução direta de consultas SQL;
- Regras de negócio.

Essa abordagem gerava baixa coesão e violava o princípio Single Responsibility Principle (SRP), dificultando a manutenção e evolução do sistema.

### Alto Acoplamento

As rotas dependiam diretamente da implementação do banco SQLite e das consultas SQL. Qualquer alteração na estrutura do banco exigia modificações em diversos pontos da aplicação.

### Baixa Escalabilidade

A arquitetura inicial funcionava adequadamente para um pequeno conjunto de funcionalidades, porém tornava-se difícil adicionar novos módulos sem aumentar ainda mais a complexidade do arquivo principal.

### Dificuldade de Integração

A estrutura existente não possuía mecanismos para integração com sistemas externos, exigindo adaptações manuais e condicionais espalhadas pelo código.

## Baixa Extensibilidade

Novas funcionalidades dependiam de alterações diretas em componentes já existentes, aumentando o risco de regressões e dificultando a aplicação do princípio **Open/Closed Principle (OCP)**.

## 2. Novos Requisitos que Motivaram a Evolução Arquitetural

Durante a segunda etapa do projeto, novos requisitos funcionais foram adicionados, exigindo uma reorganização da arquitetura.

### Compartilhamento Seguro da Conexão com Banco de Dados

Com a criação de novos serviços e módulos, múltiplas partes do sistema passaram a necessitar acesso ao banco SQLite.

O modelo anterior permitia a criação de conexões independentes, o que poderia causar:

- Travamento do arquivo SQLite (*database is locked*);
- Consumo excessivo de memória;
- Problemas de concorrência;
- Dificuldade de gerenciamento da conexão.

### Integração com E-books Externos

O sistema passou a consumir informações de uma API externa de livros digitais.

O problema identificado foi a incompatibilidade entre os formatos de dados:

| Sistema Interno | API Externa |
|-----------------|-------------|
| titulo          | name        |
| autor           | writer      |
| ano             | releaseYear |

Além disso, a API externa não possuía informações de estoque, utilizadas pelos livros físicos.

### Automatização de Processos Pós-Empréstimo

Após a realização de um empréstimo, novas ações passaram a ser obrigatórias:

- Atualização do estoque;
- Envio de confirmação ao usuário;
- Possibilidade futura de auditoria e geração de logs.

Inserir essas responsabilidades diretamente nas rotas HTTP aumentaria significativamente o acoplamento e a complexidade do sistema.

## 3. Padrões de Projeto Aplicados

### 3.1 Singleton

**Local:** `src/config/database.js`

Foi criada uma classe responsável por gerenciar a conexão com o banco SQLite através de uma única instância compartilhada em toda a aplicação.

A instância é criada apenas uma vez e disponibilizada para todos os módulos que necessitam acesso ao banco.

### 3.2 Adapter

**Local:** `src/models/itemAdapter.js`

Foi implementada a classe `EBookAdapter`, responsável por converter os dados recebidos da API externa para o formato esperado internamente pela aplicação.

O adaptador atua como uma camada de tradução entre os dois modelos de dados.

### 3.3 Observer (Publish-Subscribe)

**Locais:**

- `src/services/loanService.js`
- `src/observers/`

O serviço de empréstimos passou a atuar como um **Subject**, enquanto os módulos de estoque e notificação atuam como **Observers**.

Sempre que um empréstimo é realizado, o serviço notifica automaticamente todos os observadores cadastrados.

## 4. Justificativas Técnicas para a Escolha dos Padrões

### Singleton

#### Motivo da Escolha

O SQLite trabalha sobre um arquivo físico único. A abertura de múltiplas conexões simultâneas pode gerar bloqueios e degradação de desempenho.

#### Benefícios Obtidos

- Controle centralizado da conexão;
- Redução do consumo de memória;
- Prevenção de múltiplas instâncias desnecessárias;
- Menor risco de conflitos de acesso ao banco.

#### Impacto em SOLID

Contribui para melhor organização da infraestrutura da aplicação, concentrando a responsabilidade de acesso ao banco em um único componente.

### Adapter

#### Motivo da Escolha

O frontend já consumia um formato de dados consolidado. Alterar toda a camada de apresentação para suportar diferentes formatos aumentaria a complexidade do sistema.

#### Benefícios Obtidos

- Isolamento da integração externa;
- Reutilização do frontend existente;
- Redução de condicionais espalhadas pelo código;
- Uniformização da representação dos itens da biblioteca.

#### Impacto em SOLID

Favorece o princípio **Open/Closed Principle (OCP)**, permitindo integrar novas fontes de dados sem modificar o código consumidor.

### Observer

#### Motivo da Escolha

O processo de empréstimo passou a gerar múltiplas ações secundárias independentes.

Sem o padrão Observer, a lógica de empréstimo ficaria responsável por executar diretamente todas essas tarefas.

### **Benefícios Obtidos**

- Desacoplamento entre eventos e ações executadas;
- Facilidade para adicionar novas funcionalidades;
- Melhor separação de responsabilidades;
- Maior reutilização dos componentes.

### **Impacto em SOLID**

- Atende ao **Single Responsibility Principle (SRP)**, mantendo o serviço de empréstimo focado apenas no processo de empréstimo.
- Atende ao **Open/Closed Principle (OCP)**, permitindo adicionar novos observadores sem modificar o serviço principal.

## **5. Impactos Arquiteturais Após a Refatoração**

A adoção dos padrões de projeto trouxe melhorias significativas para a qualidade arquitetural do sistema.

### **Redução de Acoplamento**

Os módulos passaram a depender de abstrações e responsabilidades específicas, reduzindo dependências diretas entre componentes.

### **Aumento da Coesão**

Cada componente passou a possuir uma responsabilidade bem definida:

- Banco de dados → gerenciamento da conexão;
- Adaptador → conversão de dados externos;
- Serviço de empréstimos → execução do empréstimo;
- Observadores → ações secundárias.

### **Maior Extensibilidade**

Novas funcionalidades podem ser adicionadas com impacto mínimo no código existente.

Exemplos:

- Novo tipo de integração externa utilizando outro Adapter;
- Novos observadores para auditoria, relatórios ou métricas;
- Novos serviços reutilizando a mesma conexão Singleton.